

## Highlights

### **TS-PINNs: Physics-Informed Neural Networks in Temporal Sobolev Spaces**

Mingkun Li, Xincan Yu, Xiaoxin Wang, Han Zhu, Hong-Kun Zhang

- A principled approach to temporal regularization in PINNs using temporal Sobolev norms, enhancing both accuracy and physical fidelity.
- A multi-stage training procedure that stabilizes learning and enables scalable enforcement of higher-order constraints.
- Theoretical justification grounded in Sobolev approximation theory and empirical validation on benchmark PDEs.
- Demonstrated improvements over classical PINNs and recent variants in accuracy, robustness, and convergence.

# TS-PINNs: Physics-Informed Neural Networks in Temporal Sobolev Spaces

Mingkun Li<sup>a,b,c</sup>, Xincan Yu<sup>a,d</sup>, Xiaoxin Wang<sup>a,b,c,e</sup>, Han Zhu<sup>c</sup>, Hong-Kun Zhang<sup>c,\*</sup>

<sup>a</sup>Great Bay Institute for Advanced Study, Dongguan, 523000, Guangdong, China

<sup>b</sup>University of Science and Technology of China, Hefei, 230026, Anhui, China

<sup>c</sup>School of Sciences, Great Bay University, Dongguan, 523000, Guangdong, China

<sup>d</sup>Department of Mathematics, Sun Yat-sen University, Guangzhou, 510275, Guangdong, China

<sup>e</sup>Guangdong Provincial Key Laboratory of Mathematical and Neural Dynamical Systems, Dongguan, 523000, Guangdong, China

---

## Abstract

We propose TS-PINNs, a novel class of physics-informed neural networks that incorporate temporal Sobolev regularization to enhance the accuracy, stability, and generalization of neural PDE solvers. Unlike standard PINNs that penalize residuals only at discrete points, TS-PINNs leverage the structure of Sobolev spaces to enforce smoothness in time by explicitly regularizing higher-order temporal derivatives of the solution. This approach improves numerical stability, mitigates error accumulation in long-time integration, and leads to physically consistent approximations for time-dependent differential equations.

The TS-PINN framework is implemented in a multi-stage training procedure, where earlier stages learn temporally coarse solutions, and later stages refine them using higher-order constraints derived from the governing PDE. We provide theoretical motivation based on functional approximation theory in Sobolev spaces, and we empirically validate the method on several benchmark problems, including the Burgers equation and nonlinear wave propagation. Across all tasks, TS-PINNs demonstrate improved accuracy, enhanced robustness to noise, and faster convergence relative to classical PINNs and recent variants. Our results show that temporal Sobolev regularization provides a principled and practical enhancement to physics-informed learning of time-evolving systems.

**Keywords:** Neural Networks, Temporal-Sobolev Physics-informed neural networks, Physics-informed neural networks, Differential equations

---

\*Corresponding author. E-mail address: zhanghk@gbu.edu.cn.

---

## 1. Introduction

In recent years, neural networks have emerged as powerful tools for solving complex differential equations, especially in the context of physics-informed neural networks (PINNs) [1, 2, 3]. PINNs leverage the flexibility of neural networks to integrate governing physical laws directly into the loss function, enabling the efficient and accurate solution of differential equations across a range of applications in battery degradation [4], quantum optics [5], fluid mechanics [6, 7, 8], solid mechanics [9, 10].

Efforts have been made to improve traditional PINNs. Several approaches have been proposed to improve the training efficiency, accuracy, and overall performance of PINNs [11, 12, 13, 14, 15, 16, 17, 18]. While PINNs have demonstrated significant success in solving a variety of differential equations, they face several inherent limitations.

- 1) PINNs may struggle near complicated boundary conditions, especially when these conditions exhibit strong nonlinearity, leading to challenges in accurately modeling the behavior of the system at the boundaries.
- 2) The loss of regularity in the high-order derivatives of the solution approximation by PINNs often results in reduced accuracy, particularly in cases where higher-order derivatives are crucial for capturing the full dynamics of the system.

These challenges arise from the reliance of traditional PINNs on lower-order derivatives and their simple, direct training process, which may not be sufficiently sophisticated to fully capture the intricate dynamics of more complex systems. Specifically, for problems involving high-order temporal or spatial derivatives, or boundary conditions that exhibit significant nonlinearity, traditional PINNs can struggle to provide precise and reliable approximations. The standard PINNs may struggle to accurately approximate high-order derivatives of the solution, especially when the solution exhibits strong oscillations or rapid variations in time or space. This highlights the need for an enhanced approach that can better address these complex scenarios and improve the overall performance of the network.

To address the limitations of traditional PINNs, we propose novel **unsupervised** extension termed **Temporal Sobolev Physics-Informed Neural Networks**

(TS-PINNs). This framework enhances the standard PINN methodology by incorporating higher-order temporal derivatives into the loss function. By penalizing discrepancies in higher-order derivatives, TS-PINNs offer a more accurate and robust representation of dynamical systems where such terms are essential.

The approach is grounded in the Sobolev space framework, allowing the neural network to approximate not only the solution of the differential equation but also its derivatives up to a prescribed order. By leveraging a discrete version of the Sobolev norm, TS-PINNs promote smoothness and stability, yielding reliable solutions even for systems characterized by complex or high-order dynamics. Furthermore, we establish a universal approximation theorem to guarantee the convergence of the TS-PINN architecture.

Furthermore, the training process of TS-PINNs is divided into three stages, with the first stage focusing on pretraining the boundary and initial conditions. This step ensures that the model performs well near the boundary, effectively capturing the boundary dynamics and improving the overall accuracy of the solution in these critical regions. TS-PINNs are applied in this paper to demonstrate their ability to accurately solve differential equations that involve higher-order time derivatives, i.e., the Korteweg–De Vries (KdV) equation [? ? ], the heat equation [? ], the Burgers equation [? ] and the Gray-Scott equation [? ? ]. Through experiments, we compare the performance of TS-PINNs with traditional PINNs and demonstrate the advantages of using higher-order derivatives for achieving better accuracy and stability.

The structure of this paper is organized as follows. In Section 2, we describe the works related to our paper and the motivation of this paper. In Section 3, we introduce the TS-PINNs, which we propose as an extension to traditional PINNs. This section includes a detailed formulation of TS-PINNs and presents a universal approximation theorem that ensures the effectiveness of our approach. Section 4 gives a universal approximation theorem for the TS-PINNs. In Section 5, we conduct a series of experiments on the KdV equation, the heat equation, the Burgers equation and the Gray-Scott equation. These experiments compare the performance of traditional PINNs and TS-PINNs, demonstrating that our TS-PINNs significantly outperform the conventional PINNs. Finally, Section 6 concludes the paper by summarizing our findings and outlining potential directions for future work.

## 2. Backgrounds

### 2.1. Related Works

Over the past few years, significant advancements have been made to address the limitations of traditional Physics-Informed Neural Networks (PINNs). These efforts have resulted in a variety of strategies aimed at enhancing the training efficiency, accuracy, and overall robustness of PINNs. Broadly speaking, these strategies can be classified into two main categories: (i) redesigning the neural network architecture to better capture the underlying physics, and (ii) improving optimization techniques to mitigate training difficulties and accelerate convergence.

*Redesigning the Neural Network Architecture of PINNs.* Several architectural innovations have been proposed to increase the expressivity of PINNs and better incorporate physical priors. For example, [?] introduced adaptive activation functions that dynamically adjust during training, improving convergence and accuracy. Similarly, [?] explored novel activation functions tailored to tackle stiff and oscillatory problems. In [?], a gradient-enhanced PINN architecture was proposed, which improves the approximation of PDE solutions by explicitly incorporating gradient information in the loss function. The work by [?] introduced parameterized PINNs, enabling generalization across different instances of parameterized PDEs. Additionally, [?] proposed separable neural networks that exploit low-rank structures in high-dimensional problems to enhance scalability.

*Improved Optimization Techniques.* Many optimization-focused strategies aim to address the instability and poor convergence typically observed in traditional PINNs. These challenges arise due to factors such as PDE stiffness, gradient pathologies, and unbalanced loss terms. Various works have introduced modifications to the training process to mitigate these issues. For example, [?] employed a soft attention mechanism that adapts the loss weighting dynamically, enabling the network to focus on the most informative training points. In [?], the authors analyzed gradient flow pathologies in PINNs and proposed methods to stabilize training. Similarly, [?] identified failure modes in PINNs and recommended techniques like loss weighting and input normalization for improving convergence. The work in [?] applied neural tangent kernel theory to understand training failures and suggested adjustments to activation functions and initialization schemes. Furthermore, methods such as contrastive learning [?] and spectral deferred correction [?] have been utilized to address stiffness and training inefficiency. Learning rate annealing strategies [?] have also been shown to stabilize the training dynamics, and alternative loss formulations and optimization algorithms [?] demonstrate

how well-designed objective functions and solvers can improve the robustness and accuracy of PINNs.

## 2.2. Motivations

To illustrate the benefits of Temporal Sobolev Physics-Informed Neural Networks (TS-PINNs), we begin with a simple toy model described by the differential equation

$$u'(t) = 2t + 3t^2 + 10 \cos(100t), \quad (2.1)$$

subject to the initial condition  $u(0) = 0$ .

We apply traditional PINNs to approximate the solution by incorporating both the differential equation (2.1) and the initial condition into the loss function. In contrast, during the training of TS-PINNs—whose methodology is detailed in Section 3—we introduce an additional penalty term involving the second-order temporal derivative  $u''(t)$  to enforce smoothness. Specifically, we use the known expression

$$u''(t) = 2 + 6t - 1000 \sin(100t)$$

as part of the loss function during training.

Specifically, the time interval considered in the experiment is  $t \in [0, 10]$ . A total of 100 collocation points are used, which are uniformly distributed over the domain. Both PINN and TS-PINN models employ a fully connected neural network with one hidden layer consisting of 32 neurons. The models are trained for 100,000 epochs. We then compare the performance of standard PINNs and TS-PINNs in approximating the solution to this differential equation. The results of this comparison are shown in Figure 1, where it is evident that incorporating the higher-order derivative term improves accuracy by suppressing spurious oscillations and aligning the solution with the underlying dynamics of the system.

To further elucidate this effect, we turn to the Taylor expansion of a smooth function. Consider a function  $f : \mathbb{R} \rightarrow \mathbb{R}$  that is  $C^k$ -smooth at some point  $t_0$ . Then, there exists a remainder function  $r_k : \mathbb{R} \rightarrow \mathbb{R}$  with  $\lim_{t \rightarrow t_0} r_k(t) = 0$  such that

$$f(t) = f(t_0) + f'(t_0)(t - t_0) + \cdots + \frac{f^{(k)}(t_0)}{k!}(t - t_0)^k + r_k(t)(t - t_0)^k.$$

For a fixed residual point  $t_0$  in the set of residual points, the neural network  $\hat{u}(t; \theta)$  can be expanded in a similar form:

$$\hat{u}(t; \theta) = \hat{u}(t_0; \theta) + \partial_t \hat{u}(t_0; \theta)(t - t_0) + \cdots + \frac{1}{k!} \partial_t^k \hat{u}(t_0; \theta)(t - t_0)^k + r_k(t; \theta)(t - t_0)^k. \quad (2.2)$$

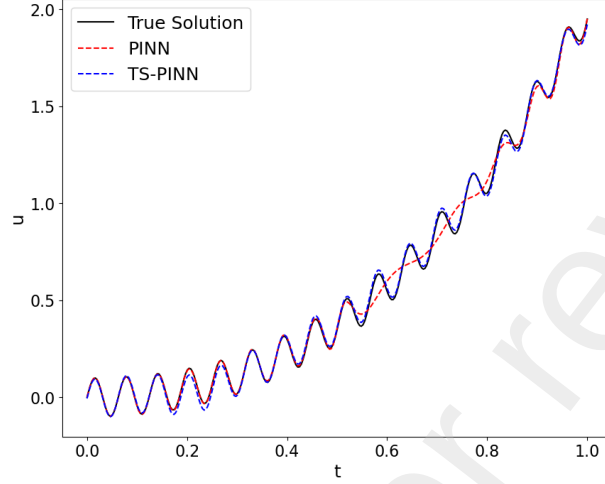


Figure 1: Comparison of the approximate solutions: PINNs (red) fail to accurately approximate the true solution (black), even at residual points, when higher-order derivatives are not considered. In contrast, TS-PINNs (blue) provide an accurate approximation.

Since the neural network is trained to minimize the PDE residual at a finite set of collocation points, the learned solution  $\hat{u}(t; \theta)$  is expected to approximate the true solution reasonably well at these locations. However, as illustrated in Figure 1, traditional PINNs may exhibit satisfactory accuracy at the residual points while still producing large deviations from the true solution in the regions between them. This phenomenon indicates that pointwise enforcement of the PDE residual alone does not sufficiently control the global smoothness or regularity of the learned solution, which may lead to reduced accuracy and poor generalization.

By explicitly incorporating higher-order time derivatives into the loss function, TS-PINNs introduce additional structural constraints on the neural network. These constraints restrict the admissible hypothesis space by enforcing consistency not only at the level of the solution itself, but also at the level of its temporal derivatives. From this perspective, the inclusion of higher-order derivatives acts as a form of regularization that enhances smoothness and coherence of the learned solution, rather than relying solely on pointwise residual minimization.

An important consequence of this formulation is the improved stability of TS-PINNs. Here, stability refers to the ability of the method to maintain comparable accuracy under different choices of residual point sampling. From the Taylor ex-

pansion in (2.2), one observes that, once temporal derivatives up to order  $k$  are constrained, the local approximation accuracy of order  $k$  is preserved independently of the specific distribution of residual points. As a result, TS-PINNs are less sensitive to sampling variability than traditional PINNs, whose performance may strongly depend on the placement of collocation points.

In conclusion, from the toy model, TS-PINNs offer not only improved accuracy but also enhanced stability, making them a more robust and effective method for solving differential equations.

### 3. Temporal Sobolev Physics-Informed Neural Networks (TS-PINNs)

In scientific research, models are often implicitly constructed based on physical laws or patterns established through prior studies, while the temporal evolution or spatial distribution of variables constrained by these laws remains to be determined. Such scenarios are typically described by differential equations governing a multivariate function  $\mathbf{u}(t, \mathbf{x}) = (u^1(t, \mathbf{x}), \dots, u^N(t, \mathbf{x}))$ , where  $t \in \mathbb{R}$  represents time and  $\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^d$  denotes the spatial coordinates. Here, the positive integer  $N \in \mathbb{Z}_+$  represents the number of dependent variables, while  $d \in \mathbb{Z}_+$  specifies the spatial dimension.

In this paper, we introduce a novel, unsupervised neural network architecture, the Temporal Sobolev Physics-Informed Neural Network (TS-PINN), to solve the differential equation (3.1), along with the associated initial conditions (3.2) and boundary conditions (3.3). The TS-PINN framework builds on the traditional PINN approach, incorporating higher-order temporal derivatives into the loss function to more accurately capture the underlying dynamics of the system.

#### 3.1. Problem Statement

We begin by providing a detailed statement of the problem we aim to solve.

Let  $K \in \mathbb{Z}_+$  be a positive integer. The problem considered in this article is to find the solution of a  $K$ -th order differential equation, expressed as:

$$\mathcal{F}(t, \mathbf{x}; \mathbf{u}; \mathbf{D}_{i=1}^K \mathbf{u}) = 0, \quad t \in I, \quad \mathbf{x} \in \Omega, \quad (3.1)$$

subject to the initial conditions:

$$\mathcal{I}(\mathbf{x}; \mathbf{u}|_{t=0}; \mathbf{D}_{i=1}^K \mathbf{u}|_{t=0}) = 0, \quad \mathbf{x} \in \Omega, \quad (3.2)$$

and the boundary conditions:

$$\mathcal{B}(t, \mathbf{x}; \mathbf{u}; \mathbf{D}_{i=1}^K \mathbf{u}) = 0, \quad t \in I, \quad \mathbf{x} \in \partial\Omega, \quad (3.3)$$

where  $I$  is an open subset of  $\mathbb{R}$  and  $\Omega$  is an open subset of  $\mathbb{R}^d$ .

For simplicity, we introduce  $\mathbf{D}_{i=1}^K \mathbf{u}$  to represent all derivatives of  $\mathbf{u}$  up to order  $K$ :

$$\mathbf{D}_{i=1}^K \mathbf{u} := (\partial_t \mathbf{u}, \partial_{\mathbf{x}} \mathbf{u}; \dots; \partial_t^K \mathbf{u}, \partial_t^{K-1} \partial_{\mathbf{x}} \mathbf{u}, \dots, \partial_t \partial_{\mathbf{x}}^{K-1} \mathbf{u}, \partial_{\mathbf{x}}^K \mathbf{u}),$$

where the mixed partial derivative  $\partial_t^{k_1} \partial_{\mathbf{x}}^{k_2} \mathbf{u}$  represents the vector generated by

$$\{\partial_t^{k_1} \partial_{\mathbf{x}}^{\alpha} \mathbf{u} \mid |\alpha| = k_2\},$$

for non-negative integers  $k_1, k_2 \in \mathbb{Z}_{\geq 0}$  satisfying  $1 \leq k_1 + k_2 \leq K$ . Here, for a multi-index  $\alpha = (\alpha_1, \dots, \alpha_d)$ , the partial derivative operator is given by

$$\partial_{\mathbf{x}}^{\alpha} = \left( \frac{\partial}{\partial x_1} \right)^{\alpha_1} \cdots \left( \frac{\partial}{\partial x_d} \right)^{\alpha_d},$$

where  $|\alpha| = \sum_{i=1}^d \alpha_i$  denotes the total order of differentiation.

In many practical applications involving differential equations and physical laws, the functions  $\mathcal{F}$ ,  $\mathcal{I}$ , and  $\mathcal{B}$  are not necessarily scalar-valued but can instead be vector-valued functions.

Solving the differential equation is essential, as it reveals the explicit dynamics or distributions that characterize the model. However, in most cases, determining the exact solution  $\mathbf{u}(t, \mathbf{x})$  of these equations is highly challenging. Therefore, numerical methods are often required to provide an accurate approximation of the solution.

We unify the treatment of PDEs (when  $d \geq 1$ ) and ODEs (when  $d = 0$ ) by representing both within the general framework of equations (3.1), (3.2), and (3.3). This unified perspective allows us to address a broad spectrum of problems under a single formulation. Specifically, we consider the approximation of solutions to these equations using neural networks, which serve as a flexible and powerful tool for capturing the underlying dynamics of both PDEs and ODEs. By leveraging this framework, we aim to explore the neural network's capability to approximate solutions with high accuracy and efficiency, regardless of whether the governing equation involves spatial and temporal variables (PDEs) or temporal variables alone (ODEs). This approach not only simplifies the methodology but also provides a consistent basis for analyzing and comparing neural network performance across diverse applications.

### 3.2. Sobolev Space and Temporal Sobolev Space

A key limitation of PINNs is their reliance on the differential equation itself during the training process. While this approach is sufficient for approximating

low-order derivatives, it becomes problematic when attempting to compute high-order derivatives of the target functions.

From the perspective of norms used in the training process, the  $L^2$  norm is commonly employed in the formulation of the loss functions of PINNs, which tends to focus solely on minimizing the loss associated with the differential equation, without explicitly considering the precision of high-order derivatives. While effective in many cases, the  $L^2$  norm performs poorly when it comes to approximating high-order derivatives. To address this limitation, a norm that simultaneously constrains both the differential equation and its high-order derivatives is needed in the loss function formulation. This would allow for better approximation of high-order derivatives, ensuring more accurate and stable solutions, especially in problems where such derivatives are crucial.

To describe the mathematical principles of our TS-PINNs, we first introduce the definition of Sobolev spaces, as first defined by S. L. Sobolev [? ? ].

**Definition 1** (Sobolev Space). *Let  $k, p \in \mathbb{Z}_{\geq 0}$  be non-negative integers. A Sobolev space with differentiability order  $k$  and integrability order  $p$  on an open set  $U \subset \mathbb{R}^D$  is the space of functions  $f$  defined on  $U$  such that the  $p$ -th power of the absolute value of  $f$  and its derivatives up to order  $k$  are integrable, where  $1 \leq k \leq \infty$ . The Sobolev space of differentiability  $k$  and integrability  $p$  on the open set  $U$  is denoted by  $W^{k,p}(U)$ . The norm of a function  $f \in W^{k,p}(U)$  is defined as the sum of the  $L^p$  norms of its derivatives up to order  $k$ , and is given by:*

$$\|f\|_{k,p} = \sum_{|\beta| \leq k} \|\partial^\beta f\|_{L^p(U)},$$

where  $\partial^\beta$  represents the partial derivatives of  $f$  up to order  $k$ .

The temporal Sobolev space is designed to capture the smoothness of a function's temporal derivatives, requiring the boundedness of these derivatives. This extension of the Sobolev space provides a more powerful tool for estimating  $C^k$ -smooth functions. Specifically, we define the temporal Sobolev space as follows:

**Definition 2** (Temporal Sobolev Space). *Let  $l, q, k \in \mathbb{Z}_{\geq 0}$  be non-negative integers. Assume that  $I$  is an open subset of  $\mathbb{R}$  and  $\Omega$  is an open subset of  $\mathbb{R}^d$ . The temporal Sobolev space  $W_k^{l,q}(I \times \Omega)$  on the open set  $I \times \Omega$  is the set of all functions  $f(t, \mathbf{x}) \in W^{l,q}(I \times \Omega)$  such that for any index  $\beta$  with  $|\beta| \leq k$ , the temporal partial derivative  $\partial_t^\beta f$  exists and  $\partial_t^\beta f \in W^{l,q}(I \times \Omega)$ .*

The temporal Sobolev space  $W_k^{l,q}(I \times \Omega)$  is a normed space. In fact, the temporal Sobolev norm of order  $k$  on the time interval  $I$  can be written as:

$$\|f\|_{W_k^{l,q}(I \times \Omega)} = \sum_{\beta=0}^k \|\partial_t^\beta f\|_{W^{l,q}(I \times \Omega)} \quad (3.4)$$

It is required to show that  $\|\cdot\|_{W_k^{l,q}(I \times \Omega)}$  is a norm.

**Proposition 3.** *Let  $W_k^{l,q}(I \times \Omega)$  be a temporal Sobolev space defined on an open set  $I \times \Omega$ , then  $\|\cdot\|_{W_k^{l,q}(I \times \Omega)}$  defined by (3.4) is a norm.*

*Proof.* We will show the positive definiteness, absolute homogeneity, and triangle inequality of the function  $\|\cdot\|_{W_k^{l,q}(I \times \Omega)} : W_k^{l,q}(I \times \Omega) \rightarrow \mathbb{R}$ .

If  $f = 0$ , from the definition, we know that  $\|f\|_{W_k^{l,q}(I \times \Omega)} = 0$ . Conversely, if  $\|f\|_{W_k^{l,q}(I \times \Omega)} = 0$ , then  $\|f\|_{W^{l,q}(I \times \Omega)} = 0$ , and thus  $f = 0$  by the property of Sobolev norms.

Given  $s \in \mathbb{R}$ , we have:

$$\begin{aligned} \|sf\|_{W_k^{l,q}(I \times \Omega)} &= \sum_{\beta=0}^k \|\partial_t^\beta(sf)\|_{W^{l,q}(I \times \Omega)} \\ &= \sum_{\beta=0}^k |s| \|\partial_t^\beta f\|_{W^{l,q}(I \times \Omega)} = |s| \sum_{\beta=0}^k \|\partial_t^\beta f\|_{W^{l,q}(I \times \Omega)} \\ &= |s| \cdot \|f\|_{W_k^{l,q}(I \times \Omega)}, \end{aligned}$$

which shows absolute homogeneity.

By the triangle inequality of the Sobolev norm, for  $f_1, f_2 \in W_k^{l,q}(I \times \Omega)$ ,

$$\begin{aligned} \|f_1 + f_2\|_{W_k^{l,q}(I \times \Omega)} &= \sum_{\beta=0}^k \|\partial_t^\beta(f_1 + f_2)\|_{W^{l,q}(I \times \Omega)} \\ &\leq \sum_{\beta=0}^k \|\partial_t^\beta f_1\|_{W^{l,q}(I \times \Omega)} + \sum_{\beta=0}^k \|\partial_t^\beta f_2\|_{W^{l,q}(I \times \Omega)} \\ &= \|f_1\|_{W_k^{l,q}(I \times \Omega)} + \|f_2\|_{W_k^{l,q}(I \times \Omega)}. \end{aligned}$$

which shows the triangle inequality.  $\square$

### 3.3. Neural Network Construction

The TS-PINN framework is designed to solve the differential equation (3.1) subject to the initial conditions (3.2) and boundary conditions (3.3). To achieve this, a neural network, such as a multi-layer perceptron (MLP) or a convolutional neural network (CNN), parameterized by  $\theta$ , is employed to approximate the true solution. The network produces an output  $\hat{\mathbf{u}}(t, \mathbf{x}; \theta)$ , which serves as an estimate of the exact solution of the differential equation.

Using the temporal Sobolev norm, the residual of the governing equation should vanish in the temporal Sobolev norm:

$$\|\mathcal{F}(t, \mathbf{x}; \hat{\mathbf{u}}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}})\|_{W_k^{l,q}(I \times \Omega)} = 0. \quad (3.5)$$

To construct the loss function, equation (3.5) is discretized, and a set of scattered points  $\mathcal{T}$ , referred to as residual points, is sampled within the domain. These residual points are strategically categorized into three subsets based on their roles in enforcing the differential equation and its associated constraints:

- 1)  $\mathcal{T}_{\mathcal{F}} \subset I \times \Omega$ : A set of points located within the interior of the domain, where the governing differential equation is enforced.
- 2)  $\mathcal{T}_I \subset \Omega$ : A collection of points at the initial time, used to enforce the initial conditions.
- 3)  $\mathcal{T}_{\mathcal{B}} \subset \partial I \times \Omega$ : A set of points situated on the boundary of the domain, used to enforce the boundary conditions.

To train the TS-PINN effectively, a suitable loss function must be defined to ensure that the model output  $\hat{\mathbf{u}}(t, \mathbf{x}; \theta)$  closely approximates the true solution. A key component of this loss function is a novel term designed to enforce smoothness and improve solution accuracy. This new loss function term is formulated as follows, defined over the set of residual points  $\mathcal{T}_{\mathcal{F}}$ :

$$\mathcal{L}_k(\theta; \mathcal{T}_{\mathcal{F}}) = \frac{1}{|\mathcal{T}_{\mathcal{F}}|} \sum_{\beta=1}^k \sum_{(t, \mathbf{x}) \in \mathcal{T}_{\mathcal{F}}} \lambda_{\beta} \left| \frac{\partial^{\beta}}{\partial t^{\beta}} (\mathcal{F}(t, \mathbf{x}; \hat{\mathbf{u}}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}})) \right|^p, \quad (3.6)$$

where  $p > 0$  is a chosen constant, and  $\lambda_{\beta} > 0$  are weighting coefficients that regulate the contribution of derivatives of different orders.

The term  $\mathcal{L}_k(\theta; \mathcal{T}_{\mathcal{F}})$  quantifies the average  $p$ -power residual error of the differential equation and its  $k$ -th order temporal derivatives at the points  $\mathcal{T}_{\mathcal{F}} \subset I \times \Omega$ ,

which lie inside the computational domain. By incorporating (3.6) into the loss function, the TS-PINN framework ensures not only pointwise accuracy but also higher-order smoothness and temporal regularity of the learned solution, which is crucial for accurately capturing complex dynamical behaviors.

Finally, the total  $k$ -th order residual error can be written as:

$$\mathcal{L}_{\mathcal{F},k}(\boldsymbol{\theta}; \mathcal{T}_{\mathcal{F}}) = \frac{1}{|\mathcal{T}_{\mathcal{F}}|} \sum_{(t,\mathbf{x}) \in \mathcal{T}_{\mathcal{F}}} |\mathcal{F}(t, \mathbf{x}; \hat{\mathbf{u}}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}})|^p + \mathcal{L}_k(\boldsymbol{\theta}; \mathcal{T}_{\mathcal{F}}). \quad (3.7)$$

Moreover, the TS-PINN output  $\hat{\mathbf{u}}(\mathbf{x}; \boldsymbol{\theta})$  is considered a good approximation of the true solution when the initial conditions and boundary conditions are satisfied in the discretized form, i.e., the following values are sufficiently small:

$$\begin{aligned} \mathcal{L}_{IC}(\boldsymbol{\theta}; \mathcal{T}_I) &= \frac{1}{|\mathcal{T}_I|} \sum_{\mathbf{x} \in \mathcal{T}_I} |\mathcal{I}(\mathbf{x}; \hat{\mathbf{u}}_{t=0}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}}_{t=0})|^2, \\ \mathcal{L}_{BC}(\boldsymbol{\theta}; \mathcal{T}_B) &= \frac{1}{|\mathcal{T}_B|} \sum_{(t,\mathbf{x}) \in \mathcal{T}_B} |\mathcal{B}(t, \mathbf{x}; \hat{\mathbf{u}}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}})|^2. \end{aligned}$$

Here  $\mathcal{L}_{IC}(\boldsymbol{\theta}; \mathcal{T}_I)$  quantifies the average squared residual error of the differential equation at the points  $\mathcal{T}_I \subset \Omega$ , which are located at the initial time;  $\mathcal{L}_{BC}(\boldsymbol{\theta}; \mathcal{T}_B)$  measures the average squared residual error of the boundary conditions at the points  $\mathcal{T}_B \subset I \times \partial\Omega$ , which are located on the boundary of the domain.

To train the TS-PINN, we define the loss function as a weighted sum of the  $p$ -power residuals evaluated at the selected points for the differential equation and the square residuals for the initial conditions and boundary conditions. Specifically, the training loss function is expressed as:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = w_{\mathcal{F}} \mathcal{L}_{\mathcal{F},k}(\boldsymbol{\theta}; \mathcal{T}_{\mathcal{F}}) + w_{IC} \mathcal{L}_{IC}(\boldsymbol{\theta}; \mathcal{T}_I) + w_{BC} \mathcal{L}_{BC}(\boldsymbol{\theta}; \mathcal{T}_B),$$

where  $w_{\mathcal{F}}, w_{IC}, w_{BC}$  are weights that control the relative importance of the residuals from the differential equation, initial conditions, and boundary conditions, respectively. The weights are typically chosen based on the specific problem being solved, aiming to balance the contributions from each term during the training process.

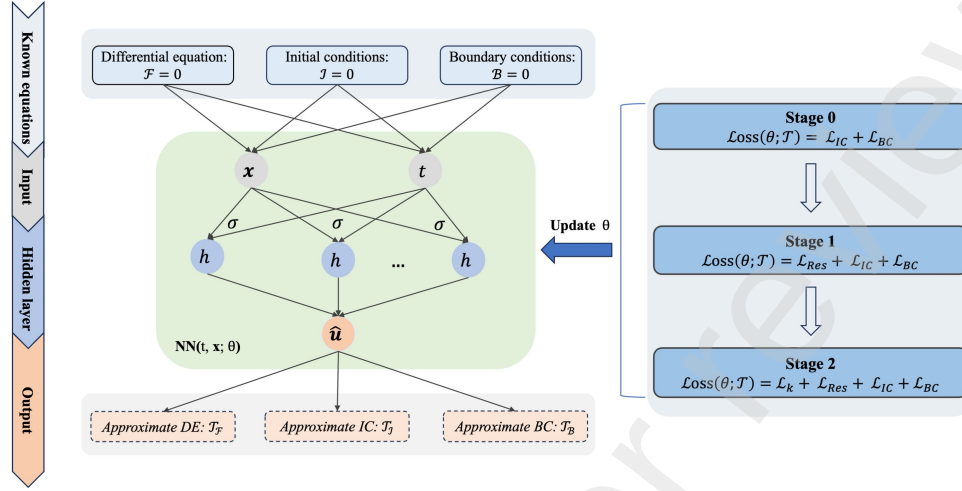


Figure 2: Schematic of a TS-PINN for solving the equation (3.1), the initial condition (3.2), and the boundary condition (3.3). In TS-PINNs, the neural network  $\hat{\mathbf{u}}(t, \mathbf{x}; \theta)$ , parameterized by  $\theta$ , is used to approximate the true solution of the underlying differential equation. The model is trained in multiple stages by minimizing the residuals of the differential equation, the initial condition, and the boundary condition. These stages will be discussed in detail in Section 5. Briefly, in **Stage 0**, the training focuses solely on fitting the initial and boundary conditions. In **Stage 1**, the differential equation residual is introduced into the training process. Specifically, the residual loss is given by  $\mathcal{L}_{\text{Res}} = \frac{1}{|\mathcal{T}_{\mathcal{F}}|} \sum_{(t, \mathbf{x}) \in \mathcal{T}_{\mathcal{F}}} |\mathcal{F}(t, \mathbf{x}; \hat{\mathbf{u}}; \mathbf{D}_{i=1}^K \hat{\mathbf{u}})|^p$ . In **Stage 2**, higher-order residual terms are incorporated into the loss function, where  $\mathcal{L}_k$  denotes the residual corresponding to the  $k$ -th order derivative.

The training process (Figure 2) involves minimizing the loss function  $\mathcal{L}(\theta; \mathcal{T})$ , which encourages the TS-PINN to align with the physical laws described by the differential equation and its boundary conditions. By doing so, the network learns to accurately approximate the true solution of the system, while incorporating the influence of both initial and boundary conditions.

By leveraging this approach, the TS-PINN offers several significant advantages. First, it not only aligns with the physical laws encoded in the differential equation (3.1), the initial condition (3.2), and the boundary condition (3.3), but also ensures accurate approximation of both the solution and its high-order derivatives. Second, the shallow architecture of the network, combined with the explicit computation of high-order derivatives, significantly reduces computational complexity compared to methods relying on purely numerical differentiation. This results in a highly efficient method that maintains precision while being compu-

tationally scalable. Consequently, the TS-PINN provides a powerful and flexible framework for solving differential equations with high accuracy and efficiency.

#### 4. Theoretical Formulation

In this subsection, we introduce the universal approximation theorem. Previous approaches have utilized polynomials [?] and trigonometric functions [?] to approximate arbitrary smooth functions. Moreover, universal approximation theorems have been given for different network structures and norms [?]. Theorem 4 guarantees the convergence of the neural network, establishing that any smooth function can be effectively approximated by TS-PINNs in the temporal Sobolev norm  $\|\cdot\|_{W_k^{l,q}(I \times \Omega)}$ . This result is crucial for the TS-PINN framework, as it ensures that both the solution of the differential equation and its higher-order derivatives can be learned with high accuracy, enabling the network to model complex temporal dynamics efficiently. By leveraging this theoretical foundation, TS-PINNs offer a robust framework for solving differential equations with high-order derivative information, making them particularly useful for problems requiring precise modeling of temporal evolution and dynamics.

The key improvement of TS-PINNs lies in their ability to approximate higher-order derivatives more accurately. The inclusion of higher-order temporal derivatives in the loss function enforces smoothness constraints, ensuring that the approximation converges in the temporal Sobolev space. This is in contrast to traditional PINNs, which use the  $L^2$ -norm and fail to enforce higher-order derivative smoothness, leading to suboptimal performance for problems requiring accurate temporal derivative approximations.

In the universal approximation theorem, the choice of the activation function plays a crucial role in determining the expressiveness and convergence properties of neural networks. For TS-PINNs, we assume that the derivative  $\sigma'$  of the activation function  $\sigma$  satisfies the following conditions:

$$\lim_{t \rightarrow +\infty} \sigma'(t) = 0 \quad \text{and} \quad \lim_{t \rightarrow -\infty} \sigma'(t) = 0. \quad (4.1)$$

This assumption ensures that the influence of neurons diminishes in the asymptotic regime, preventing uncontrolled growth of higher-order derivatives. Notably, many commonly used activation functions, such as the sigmoid function and the hyperbolic tangent function ( $\tanh$ ), naturally satisfy this requirement, making them suitable choices for TS-PINNs.

Let  $NN_{\text{TS-PINN}}$  denote the set of all TS-PINNs with activation functions satisfying (4.1). In the following, we present the universal approximation theorem

for TS-PINNs, which establishes that under suitable conditions on the activation function and network architecture, TS-PINNs can approximate any sufficiently smooth function, along with its derivatives, to arbitrary precision in the temporal Sobolev norm. This result provides a theoretical foundation for the effectiveness of TS-PINNs in solving differential equations.

**Theorem 4** (Universal Approximation Theorem for TS-PINNs). *Let  $f : M \rightarrow \mathbb{R}$  be a  $C^k$  continuous function defined on a compact domain  $M \subset \mathbb{R} \times \mathbb{R}^d$ . For any given  $\epsilon > 0$ , there exists an integer  $h \geq 1$  and a TS-PINN, denoted by  $\hat{\mathbf{u}}(t, \mathbf{x}; \boldsymbol{\theta}) \in \text{NN}_{\text{TS-PINN}}$ , parameterized by  $\boldsymbol{\theta}$ , with hidden layer width  $h$ , such that*

$$\|f(t, \mathbf{x}) - \hat{\mathbf{u}}(t, \mathbf{x}; \boldsymbol{\theta})\|_{W_k^{l,q}(I \times \Omega)} < \epsilon \quad (4.2)$$

for all points  $(t, \mathbf{x}) \in M$ .

Let  $\xi$  be an arbitrary measurable function defined on the bounded domain  $I^D \subset \mathbb{R}^D$ . By Hölder's inequality, for any  $1 \leq q_1 \leq q_2 \leq \infty$ , we have

$$\|\xi\|_{L^{q_1}(I^D)} \leq \text{Vol}(I^D)^{\frac{1}{q_1} - \frac{1}{q_2}} \|\xi\|_{L^{q_2}(I^D)}, \quad (4.3)$$

where  $\text{Vol}(I^D)$  denotes the Lebesgue measure of  $I^D$ . Consequently, it suffices to establish convergence in the  $L^2(I^D)$  norm, which serves as a representative and analytically convenient setting. For brevity, we henceforth denote  $L^2(I^D)$  simply by  $L^2$  whenever the underlying domain is clear from the context. To prove Theorem 4, we first establish a preliminary result that links Sobolev regularity to approximation error estimates.

**Theorem 5.** *Let  $I^D$  denote the  $D$ -dimensional cube  $[\xi, \eta]^D$ . Assume that the high derivatives  $\sigma^{(K)}$  of the  $C^K$ -smooth function  $\sigma$  are non-zero compactly supported continuous functions. Then, given  $f \in C^K(I^D)$  and  $\epsilon > 0$ , there exist  $M \in \mathbb{Z}$ ,  $\mathbf{C}, \mathbf{B} \in \mathbb{R}^M$ , and  $\mathbf{W} \in \mathbb{R}^{D \times M}$ , such that for  $0 \leq |\mathbf{k}_0| \leq K$ ,*

$$\|f^{(\mathbf{k}_0)}(\mathbf{z}) - S^{(\mathbf{k}_0)}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B})\|_{L^2} < \epsilon, \quad (4.4)$$

where  $S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = \sum_{j=1}^M C_j \sigma(\mathbf{W}_j \cdot \mathbf{z} + B_j)$ .

Theorem 5 can be applied to function approximation within the temporal Sobolev space. Specifically, when we set  $D = 1 + d$  and  $\mathbf{k}_0 = (k_0, 0, \dots, 0)$ , the approximation given by (4.4) takes the form:

$$\|D_t^{k_0} f(t, \mathbf{x}) - D_t^{k_0} S(t, \mathbf{x}; \mathbf{C}, \mathbf{W}, \mathbf{B})\|_{L^2} < \epsilon, \quad \text{for } 0 \leq k_0 \leq K.$$

This result guarantees that the temporal derivatives of the function can be accurately approximated, particularly at initial and boundary points. Consequently, the approximation in the temporal Sobolev norm  $\|\cdot\|_{W_t^{k,2}(I \times \Omega)}$  follows by appropriately rescaling  $\epsilon$ :

$$\|f(t, \mathbf{x}) - S(t, \mathbf{x}; \mathbf{C}, \mathbf{W}, \mathbf{B})\|_{W_t^{k,2}(I \times \Omega)} = \sum_{\beta=0}^k \|D_t^\beta f(t, \mathbf{x}) - D_t^\beta S(t, \mathbf{x}; \mathbf{C}, \mathbf{W}, \mathbf{B})\|_{L^2} < \epsilon.$$

Moreover, a sequence of compactly supported continuous functions can be used to approximate activation functions that satisfy condition (4.1). This observation ensures that the necessary conditions for Theorem 4 hold, meaning that once Theorem 5 is established, Theorem 4 follows as a direct consequence.

Before presenting the universal approximation theorem, we first provide a lemma to establish the foundational results necessary for the proof. This lemma gives the estimate of the derivatives of target functions.

**Remark 6.** *We clarify that Theorem 5 is stated and proved for neural networks with a single hidden layer, as reflected by the definition*

$$S(z; C, W, B) = \sum_{j=1}^M C_j \sigma(W_j \cdot z + B_j).$$

*This setting is standard in universal approximation results and is sufficient for establishing the desired density property.*

*For multi-layer neural networks, the result can still be recovered under mild conditions. In particular, if the composition of activation functions across layers satisfies the assumptions in Theorem 5, the theorem remains applicable by choosing trivial parameter values in the additional hidden layers (for example, setting  $W = (1, 0, \dots, 0)$  and  $B = 0$ ). With this choice, the multi-layer network effectively reduces to a single-hidden-layer representation, while preserving the universal approximation property and density in the target function space.*

**Lemma 7.** *Let  $I^D$  denote the  $D$ -dimensional cube  $[\xi, \eta]^D$ . Assume that  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  is a continuous compactly supported function satisfying  $\sigma \neq 0$ . Then the finite sums of the form*

$$S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = \sum_{j=1}^M C_j \sigma(\mathbf{W}_j \cdot \mathbf{z} + B_j)$$

*defined on  $I^D$  are dense in  $L^2(I^D)$ , where  $\mathbf{C} = (C_1, \dots, C_M)$ ,  $\mathbf{B} = (B_1, \dots, B_M) \in \mathbb{R}^M$ , and  $\mathbf{W} = (\mathbf{W}_1, \dots, \mathbf{W}_M) \in \mathbb{R}^{D \times M}$ .*

*Proof.* Consider the subspace  $V$  of  $L^2(I^D)$  consisting of functions in the form of  $S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B})$ :

$$V = \{S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) \in L^2(I^D) : \mathbf{C}, \mathbf{B} \in \mathbb{R}^M, \mathbf{W} \in \mathbb{R}^{D \times M}\}.$$

Claim that  $\bar{V} = L^2(I^D)$ . In fact, if not, by the Hahn-Banach theorem, there exists a linear functional  $\phi : L^2(I^D) \rightarrow \mathbb{R}$  satisfying  $\phi \neq 0$  and  $\phi(\bar{V}) = 0$ . By Riesz's representation theorem, there exists  $g_\phi \in L^2(I^D)$  such that

$$\phi(h) = \int_{I^D} h(\mathbf{z}) g_\phi(\mathbf{z}) d\mathbf{z}, \quad \text{for } h \in L^2(I^D).$$

Put  $S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) \in V$  into  $\phi$ , we have  $\phi(S) = 0$ . Since  $\sigma$  is a non-zero compactly supported function,  $g_\phi = 0$  almost everywhere, which implies that  $\phi = 0$ . This is a contradiction. Therefore, we conclude that  $\bar{V} = L^2(I^D)$ , and thus the finite sums of the form

$$S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = \sum_{j=1}^M C_j \sigma(\mathbf{W}_j \cdot \mathbf{z} + B_j)$$

are dense in  $L^2(I^D)$ . □

With the lemma established, we can now present the proof for the generalized universal approximation theorem.

*Proof of Theorem 5.* We provide the proof for  $D = 1$  as an example. When  $k = K$ , by Lemma 7, there exists

$$g_K(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = \mathbf{C} \cdot \sigma^{(K)}(\mathbf{W} \cdot \mathbf{z} + \mathbf{B}),$$

such that

$$\|f^{(K)}(\mathbf{z}) - g_K(\mathbf{z}; \mathbf{C}_1, \mathbf{W}_1, \mathbf{B}_1)\|_{L^2} < \epsilon.$$

Assume that  $g_k^{(K-k)}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = g_K(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B})$  for  $0 \leq k \leq K$ , then

$$\begin{aligned} & f^{(K-1)}(\mathbf{z}) - g_{K-1}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) - (f^{(K-1)}(\mathbf{z}_0) - g_{K-1}(\mathbf{z}_0; \mathbf{C}, \mathbf{W}, \mathbf{B})) \\ &= \int_{\mathbf{z}_0}^{\mathbf{z}} f^{(K)}(\mathbf{z}) - g_K(\mathbf{z}; \mathbf{C}_1, \mathbf{W}_1, \mathbf{B}_1) d\mathbf{z} \end{aligned}$$

Let  $A_k$  denote  $f^{(k)}(\mathbf{z}_0) - g_k(\mathbf{z}_0; \mathbf{C}, \mathbf{W}, \mathbf{B})$  for  $k = 0, \dots, K-1$ . Then

$$\begin{aligned} & f(\mathbf{z}) - g_{k_0}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) - \sum_{k=k_0}^{K-1} \frac{A_k}{(k-k_0)!} (\mathbf{z} - \mathbf{z}_0)^{k-k_0} \\ &= \int_{\mathbf{z}_0}^{\mathbf{z}} \cdots \int_{\mathbf{z}_0}^{\mathbf{z}} f^{(K)}(\mathbf{z}) - g_K(\mathbf{z}; \mathbf{C}_1, \mathbf{W}_1, \mathbf{B}_1) d\mathbf{z} \end{aligned}$$

Therefore, through certain scaling, we have

$$\|f(\mathbf{z}) - g_{k_0}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) - \sum_{k=k_0}^{K-1} \frac{A_k}{(k - k_0)!} (\mathbf{z} - \mathbf{z}_0)^{k-k_0}\|_{L^2} < \epsilon$$

Let  $S(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) = g_0(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B})$ , then

$$\|f(\mathbf{z}) - S^{(k_0)}(\mathbf{z}; \mathbf{C}, \mathbf{W}, \mathbf{B}) - \sum_{k=k_0}^{K-1} \frac{A_k}{(k - k_0)!} (\mathbf{z} - \mathbf{z}_0)^{k-k_0}\|_{L^2} < \epsilon.$$

We complete the proof for  $D = 1$ . For  $D > 1$ , the proof is analogous.  $\square$

## 5. Numerical Experiments

In this section, we demonstrate the effectiveness of TS-PINNs in solving differential equations. We apply TS-PINNs to four well-known problems: the Korteweg–de Vries (KdV) equation, the heat equation, the Burgers equation and the Gray-Scott equation. These examples illustrate the versatility of TS-PINNs in handling systems with varying levels of complexity and nonlinearity.

1. **KdV Equations:** The **Korteweg-de Vries (KdV) equation** is a third-order nonlinear partial differential equation:

$$u_t + 6uu_x + u_{xxx} = 0, \quad (5.1)$$

where  $u = u(t, x) \in \mathbb{R}$  is the wave profile as a function of spatial coordinate  $x$  and time  $t$ . Typically, one imposes boundary and/or initial conditions on a domain  $[0, T] \times [\xi, \eta]$ .

The Korteweg–de Vries (KdV) equation is a classical 1D wave PDE that describes the propagation of solitary waves in shallow water. Furthermore, the KdV equation is widely used in fluid dynamics and nonlinear wave theory. Its soliton solutions are of significant interest, and solving this equation accurately is a benchmark for numerical methods.

2. **Heat Equations:** The **heat equation** is a fundamental partial differential equation (PDE) used to describe the distribution of heat (or variation in temperature) in a given region over time:

$$u_t = \kappa u_{xx}, \quad 0 < x < \pi, \quad 0 < t < 1, \quad (5.2)$$

where  $u(x, t)$  represents the temperature at position  $x$  and time  $t$ , and  $\kappa$  is the thermal diffusivity coefficient.

3. **Burgers Equations:** Burgers equation is a fundamental partial differential equation that describes various physical phenomena, including shock waves and turbulence. The problem is formulated as:

$$u_t + uu_x - \nu u_{xx} = 0, \quad x \in [-1, 1], t \in [0, 1], \quad (5.3)$$

where  $u(t, x)$  is the velocity field at time  $t$  and position  $x$ , and  $\nu$  is the kinematic viscosity.

4. **Gray-Scott Equations:** The **Gray-Scott model** is a system of coupled, nonlinear reaction-diffusion partial differential equations:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + F(1 - u) \quad (5.4)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (F + K)v \quad (5.5)$$

where  $u = u(x, y, t)$  and  $v = v(x, y, t)$  are the concentrations of two chemical species as functions of spatial coordinates  $(x, y)$  and time  $t$ . The parameters  $D_u, D_v$  are diffusion coefficients,  $F$  is the feed rate, and  $K$  is the kill rate. Typically, one imposes boundary and/or initial conditions on a domain  $[t_{start}, t_{end}] \times [x_{min}, x_{max}] \times [y_{min}, y_{max}]$ .

The above examples highlight the broad applicability of TS-PINNs in solving partial differential equations (PDEs). By leveraging the ability of TS-PINNs to compute high-order derivatives precisely, we achieve efficient and accurate approximations for solutions to these equations. In the following sections, we provide detailed results and comparisons for each of these cases, demonstrating the effectiveness of the proposed method.

### 5.1. Experimental Setup and Training Details

In all numerical experiments, we employ feedforward neural networks as the underlying approximation architecture. All higher-order temporal and spatial derivatives appearing in the experiments are computed via automatic differentiation.

*Training Strategies.* The training procedure is organized into the following stages:

**Stage 0: Pre-training on initial and boundary conditions.** This stage provides a suitable initialization for the network parameters by enforcing the initial and boundary constraints, which helps stabilize subsequent training and improves convergence.

**Stage 1: Training with PDE residuals.** In this stage, the loss function incorporates both the PDE residuals and the initial/boundary conditions. This enables the network to refine the solution and accurately capture the underlying dynamics, particularly in regions where the residuals are significant.

**Stage 2: Training with higher-order temporal derivatives (TS-PINNs only).** This stage augments the loss function with soluble derivatives of the PDE residuals, allowing the network to explicitly learn higher-order time dynamics. By enforcing these additional constraints, TS-PINNs achieve improved temporal accuracy and stability, especially for dynamic systems governed by time-dependent PDEs. For a fair comparison, standard PINNs are also trained following the **Stage 1** strategy for the same number of epochs as those used in **Stage 2**.

*Optimizer and Loss Function.* The Adam optimizer was chosen due to its adaptive learning rate, which typically accelerates convergence, especially when dealing with non-linear systems like those found in TS-PINNs. To enhance training efficiency, the loss function used for TS-PINNs incorporates a temporal Sobolev norm that penalizes both the PDE residuals and the higher-order derivatives, ensuring smoothness in the learned solutions. For the KdV, heat and Burgers equations, we set  $k = 1$  in the loss term (3.7). For the Gray-Scott equations, we conduct experiments with both  $k = 1$  and  $k = 2$  in (3.7) in order to investigate the effect of incorporating higher-order derivative information. The error between the predicted solutions and the exact solution is quantified using Mean Squared Error (MSE) and Mean Absolute Error (MAE) in the numerical experiments.

## 5.2. Training Algorithm

We summarize the schematic used in the numerical experiments. The training algorithm (Algorithm 1) is provided to describe Figure 2. Let  $\theta$  denote the trainable parameters of the neural network  $\hat{\mathbf{u}}(t, \mathbf{x}; \theta)$ . All stages are optimized using the Adam optimizer with learning rate  $\eta$ . In each epoch,  $N_c$  collocation points are sampled uniformly from the space-time domain for enforcing the PDE constraints.

---

**Algorithm 1** Multi-stage Training Algorithm for PINNs and TS-PINNs

---

**Require:** Initial parameters  $\theta^{(0)}$ , learning rate  $\eta$ , number of collocation points  $N_c$ , maximum epochs  $N_0, N_1, N_2$

**Ensure:** Trained parameters  $\theta^*$

- 1: Initialize  $\theta \leftarrow \theta^{(0)}$
  - 2: **Stage 0: Pre-training on Initial and Boundary Conditions**
  - 3: **for**  $i = 1$  to  $N_0$  **do**
  - 4:     Sample initial and boundary points  $\mathcal{T}_I$  and  $\mathcal{T}_B$
  - 5:     Compute loss  $\mathcal{L}_{IC}(\theta; \mathcal{T}_I) + \mathcal{L}_{BC}(\theta; \mathcal{T}_B)$
  - 6:     Update parameters  $\theta$  using Adam optimizer
  - 7: **end for**
  - 8: **Stage 1: Training with PDE Residuals (PINN)**
  - 9: **for**  $i = 1$  to  $N_1$  **do**
  - 10:     Sample  $N_c$  collocation points in the space-time domain
  - 11:     Compute loss  $\mathcal{L}_{\mathcal{F};0}(\theta; \mathcal{T}_{\mathcal{F}}) + \mathcal{L}_{IC}(\theta; \mathcal{T}_I) + \mathcal{L}_{BC}(\theta; \mathcal{T}_B)$  including PDE residuals
  - 12:     Update parameters  $\theta$  using Adam optimizer
  - 13: **end for**
  - 14: **Stage 2: Training with Time-Sobolev Residuals**
  - 15: **for**  $i = 1$  to  $N_2$  **do**
  - 16:     Compute loss  $\mathcal{L}_{\mathcal{F};k}(\theta; \mathcal{T}_{\mathcal{F}}) + \mathcal{L}_{IC}(\theta; \mathcal{T}_I) + \mathcal{L}_{BC}(\theta; \mathcal{T}_B)$  including higher-order time derivatives
  - 17:     Update parameters  $\theta$  using Adam optimizer
  - 18: **end for**
  - 19: **return**  $\theta^* \leftarrow \theta$
- 

### 5.3. Results and Discussion

The detailed results of these experiments, including the performance comparison of TS-PINNs against traditional PINNs and the impact of pre-training, are presented in the following sections.

#### 5.3.1. KdV Equations

This experiment aims to solve the KdV equation using PINNs and TS-PINNs. The KdV equation is a nonlinear partial differential equation given by (5.1). The

boundary conditions for the experiment are as follows:

$$u(t, -5) = \frac{2}{\cosh^2(-5 - 4t)}, \quad u(t, 5) = \frac{2}{\cosh^2(5 - 4t)}, \quad \forall t \in [0, 2]$$

The initial condition is given by the KdV soliton solution:

$$u(0, x) = \frac{2}{\cosh^2(x)}$$

where  $x \in [-5, 5]$ .

In the experiments for the KdV equation, we employ a feedforward neural network with a single hidden layer consisting of 128 neurons. The hyperbolic tangent (tanh) activation function is used throughout the network. Training is performed using the Adam optimizer with the learning rate of  $10^{-3}$ .

We discretize the space  $x$  over the range  $[-5, 5]$  and time  $t$  over  $[0, 2]$  into grids uniformly for collocation points. The numbers of initial, boundary, and interior collocation points are set to 256, 320, and 512, respectively, all sampled uniformly within their corresponding domains. In addition, 4000 validation points are selected uniformly over the spatio-temporal domain using an equidistant grid.

In the loss function, the weights associated with the differential equation residual, the initial condition, and the boundary conditions are all set to 1. The weight corresponding to the first-order temporal derivative term is chosen as 0.1.

For TS-PINNs, the training process consists of 20000 epochs for Stage 0, 20000 epochs for Stage 1, and 30000 epochs for Stage 2. For comparison, the standard PINN is trained for 20000 epochs in Stage 0 followed by 50000 epochs in Stage 1. The analytical soliton solution of the KdV equation is used as the reference (ground-truth) solution throughout the experiments.

The error maps and slices provide insight into the accuracy of both the PINN and TS-PINN models. Both models show a high level of accuracy, with TS-PINN demonstrating a slight improvement in capturing the soliton's dynamics over time.

Compared to PINN, the TS-PINN model shows an improvement in both metrics, reflecting a more accurate prediction. The TS-PINN model achieves a 53.4% reduction in MSE and an 18.8% reduction in MAE, demonstrating its superior accuracy compared to the baseline method (See Table 1). The TS-PINN model outperforms the PINN model by reducing both the MSE and MAE, demonstrating its effectiveness in approximating the exact solution.

We evaluate the performance of both models by comparing their solutions with the exact KdV soliton solution. The results are shown in the following figures (Figure 3 and Figure 4):

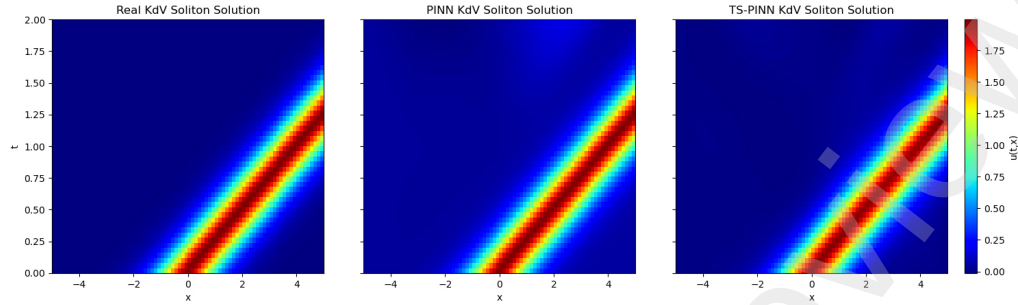


Figure 3: Comparison of the exact solution, PINN predicted solution, and TS-PINN predicted solution for the KdV equation. From left to right: (1) Exact solution, (2) PINN predicted solution, and (3) TS-PINN predicted solution. The color gradients represent the solutions of the KdV equations through different methods. The TS-PINN model provides a more accurate approximation of the exact solution compared to the PINN model, demonstrating the effectiveness of the additional penalty term in improving the solution.

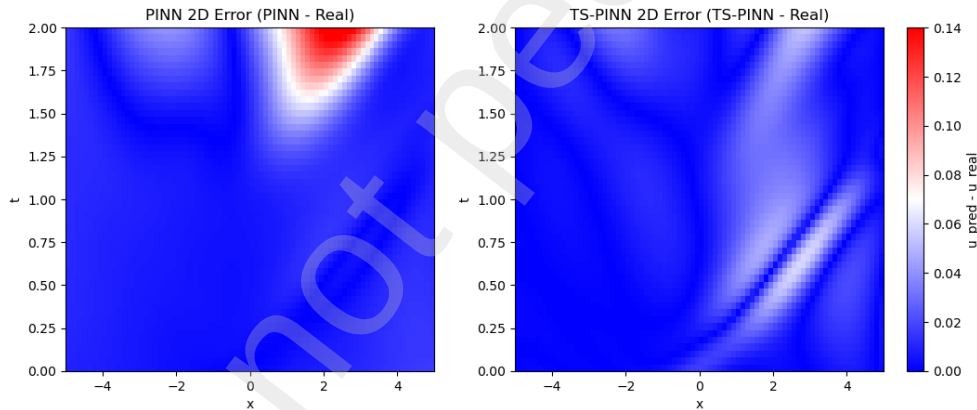


Figure 4: Error comparison between PINN and TS-PINN models with respect to the exact solution. Left: PINN 2D error (PINN - Real), Right: TS-PINN 2D error (TS-PINN - Real). The color bar represents the error magnitude, with red indicating higher errors and blue indicating lower errors. The TS-PINN model shows a significantly smaller error compared to the PINN model. The PINN error ranges from approximately 0 to 0.15, whereas the TS-PINN error ranges from 0 to 0.05, indicating a substantial improvement in prediction accuracy with TS-PINN.

The PINN approach is capable of accurately solving the KdV equation by minimizing the PDE residual over the domain. The TS-PINN model, which incorporates the second derivative with respect to time, shows further improvements in accuracy, particularly in capturing the dynamics of the KdV soliton. The error

maps highlight the areas where the models perform well and where further improvement might be necessary.

### 5.3.2. Heat Equations

We solve the heat equation (5.2) with initial and boundary conditions:

$$u(0, t) = u(\pi, t) = 0, \quad u(x, 0) = \sin(x).$$

The task is to predict the solution  $u(x, t)$  using a neural network that approximates the solution based on the governing PDE.

In the experiments for the heat equation, we employ a feedforward neural network with two hidden layers, each consisting of 64 neurons. The hyperbolic tangent (tanh) activation function is used throughout the network. Training is performed using the Adam optimizer with a learning rate of  $10^{-3}$ .

The training data consist of initial condition points  $x \in [0, \pi]$  at  $t = 0$ , boundary condition points at  $x = 0$  and  $x = \pi$  for all  $t \in [0, 1]$ , and interior collocation points sampled uniformly over the spatio-temporal domain to enforce the governing PDE. The numbers of initial, boundary, and interior collocation points are set to 2560, 6400, and 20000, respectively, all sampled uniformly within their corresponding domains. In addition, 10000 validation points are selected uniformly over the spatio-temporal domain using an equidistant grid.

In the loss function, the weights associated with the differential equation residual, the initial condition, and the boundary conditions are set to 1, 1, and 5, respectively. The weight corresponding to the first-order temporal derivative term is chosen as 0.1.

For TS-PINNs, the training process consists of 2000 epochs for Stage 0, 5000 epochs for Stage 1, and 5000 epochs for Stage 2. For comparison, the standard PINN is trained for 5000 epochs in Stage 0 followed by 5000 epochs in Stage 1. The numerical solution of the heat equation computed from the finite-element method is used as the reference (ground-truth) solution throughout the experiments.

We present the predicted solutions from both the PINN and TS-PINN models along with the exact solution. In addition, the error in the prediction is computed by comparing the numerical solutions with the exact solution.

Figure 5 shows the predicted solutions for both PINN and TS-PINN, as well as the exact solution. The predicted solutions are close to the exact solution, indicating that both models have successfully learned the underlying physics of the problem.

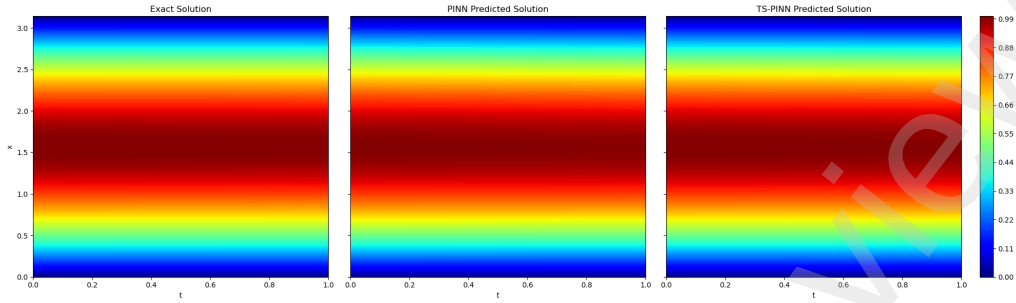


Figure 5: Comparison of the exact solution, PINN predicted solution, and TS-PINN predicted solution for the heat equation. From left to right: (1) Exact solution, (2) PINN predicted solution, and (3) TS-PINN predicted solution. The color gradient represents the temperature distribution across the spatial domain  $x \in [0, \pi]$  and temporal domain  $t \in [0, 1]$ . The TS-PINN model provides a more accurate approximation of the exact solution compared to the PINN model, demonstrating the effectiveness of the additional penalty term in improving the solution.

Figure 6 shows the error between the predicted solutions and the exact solution. The error is calculated using the absolute difference between the predicted and exact solutions at each point in the domain. The results demonstrate that the TS-PINN model yields lower errors compared to the standard PINN approach, suggesting the effectiveness of the additional penalty term in improving the model's accuracy.

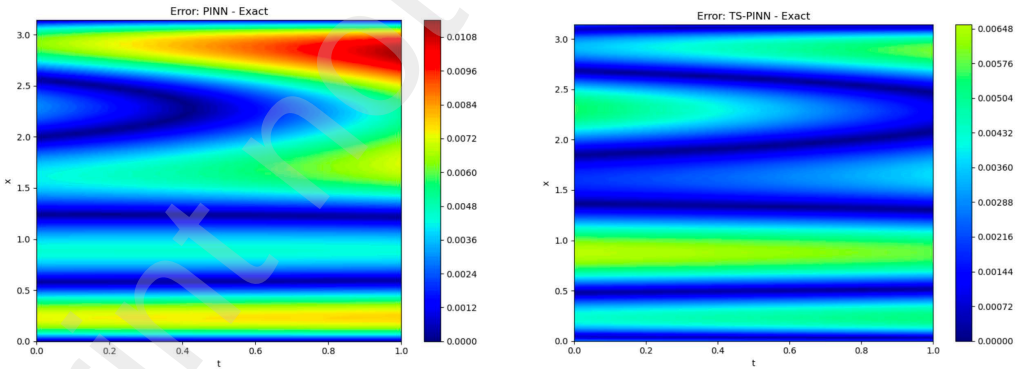


Figure 6: Error comparison between the PINN and TS-PINN models with respect to the exact solution of the heat equation. From left to right: (1) Error between the PINN predicted solution and the exact solution, (2) Error between the TS-PINN predicted solution and the exact solution. The color bar on the right of each plot indicates the magnitude of the error, with higher values represented by red and lower values by blue. The TS-PINN model results in a smaller error compared to the standard PINN model, demonstrating the improved accuracy of the TS-PINN approach.

The TS-PINN model provides approximately 40% lower error compared to the PINN model, especially in regions away from the boundaries. This indicates that the TS-PINN model is more effective in capturing the underlying physics of the heat equation. The TS-PINN model improves upon the standard PINN model by a 28.6% reduction in MAE and a 49.17% reduction in MSE compared to PINN (See table 1).

### 5.3.3. Burgers Equations

In this work, we consider the one-dimensional viscous Burgers equation (5.3) subject to the initial condition

$$u(x, 0) = -\sin(\pi x),$$

together with appropriate boundary conditions imposed at  $x = -1$  and  $x = 1$ . In all numerical experiments, the viscosity coefficient is fixed as

$$\nu = \frac{0.01}{\pi}.$$

It is worth noting that the initial profile  $-\sin(\pi x)$  exhibits a sharp transition near  $x = 0$ , which leads to steep gradients in the solution and poses a significant challenge for standard PINN approaches. To better capture the solution behavior across this interface, we adopt a region-wise training strategy by decomposing the spatial domain into two subdomains:

$$\text{Region I: } x < 0, \quad \text{Region II: } x \geq 0.$$

Correspondingly, we construct two independent neural network models,

$$u_{\text{neg}}(x, t) \quad \text{for } x < 0, \quad u_{\text{pos}}(x, t) \quad \text{for } x \geq 0,$$

each trained on its respective smooth subregion. The global approximation is then defined as the composite function

$$u(x, t) = \begin{cases} u_{\text{neg}}(x, t), & x < 0, \\ u_{\text{pos}}(x, t), & x \geq 0, \end{cases} \quad (5.6)$$

which allows the model to accurately represent the solution on both sides of the interface. Although the spatial domain is divided into two sub-domains at  $x = 0$ , continuity of the solution across the interface is assumed. Specifically, we require

that the two neural network approximations coincide at  $x = 0$ , thereby enforcing a continuity condition at the sub-domain boundary.

More generally, for partial differential equations whose solutions are *piece-wise smooth*, this region-wise training strategy provides a practical and effective extension of physics-informed learning. As demonstrated by the Burgers equation example, although singular or near-singular behavior may arise at interfaces, the solution remains smooth within each subdomain. By partitioning the computational domain accordingly and training separate models on each smooth region, the proposed method can be applied without violating the regularity assumptions required by higher-order derivative constraints.

In the experiments for the Burgers equation, we employ a feedforward neural network with two hidden layers, each consisting of 32 neurons. The hyperbolic tangent (tanh) activation function is used throughout the network. Training is performed using the Adam optimizer with a learning rate of  $10^{-3}$ .

The numbers of initial, boundary, and interior collocation points are set to 1000, 200, and 10000, respectively, all sampled uniformly within their corresponding domains. In addition, 12800 validation points are selected uniformly over the spatio-temporal domain using an equidistant grid.

In the loss function, the weights associated with the differential equation residual, the initial condition, and the boundary conditions are set to 1, 10, and 50, respectively. The weight corresponding to the first-order temporal derivative term is chosen as 0.1.

For TS-PINNs, the training process consists of 10000 epochs for Stage 0, 5000 epochs for Stage 1, and 15000 epochs for Stage 2. For comparison, the standard PINN is trained for 10000 epochs in Stage 0 followed by 20000 epochs in Stage 1. The numerical solution of the Burgers equation computed from the finite-element method is used as the reference (ground-truth) solution throughout the experiments.

The performance of the method is visualized through contour plots and three-dimensional surface plots comparing the predicted solutions against reference data. Representative results are shown in Figures 7 and 8. In addition, Figure 9 shows solution slices at different time instances, demonstrating that TS-PINNs remain accurate even in the presence of singular behavior at  $x = 0$ .

Table 1 compares the MSE and MAE for two neural PDE solvers—PINN and TS-PINN. TS-PINN achieves lower errors in both metrics, amounts to a  $\sim 27\%$  reduction in MSE and a  $\sim 75\%$  reduction in MAE when switching from PINN to TS-PINN, indicating that TS-PINN significantly improves accuracy for the Burgers equation.

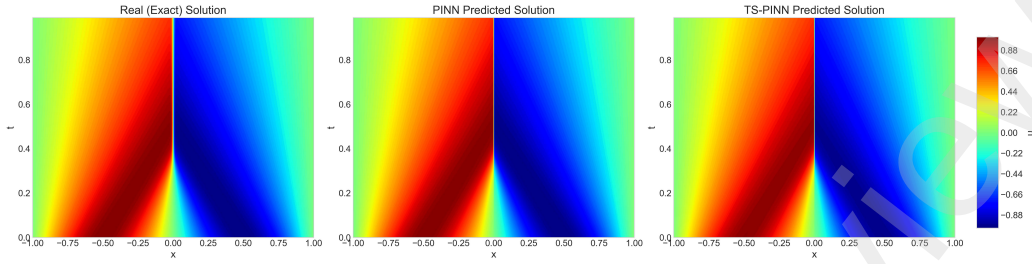


Figure 7: Comparison of solutions for the one-dimensional viscous Burgers equation on the spatiotemporal domain  $x \in [-1, 1]$ ,  $t \in [0, 1]$ . The left image shows the solution predicted by a composite physics-informed neural network (PINN), the middle image by a composite TS-PINN, and the right image the real (exact) solution generated via a high-resolution numerical scheme.

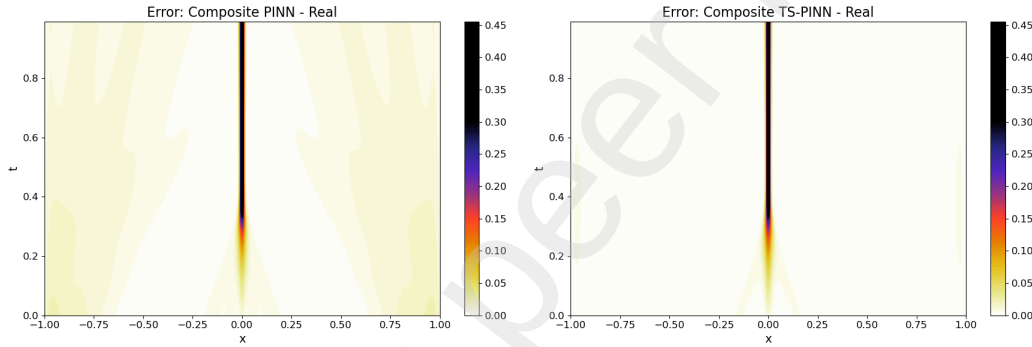


Figure 8: The horizontal axis is the spatial coordinate  $x \in [-1, 1]$  and the vertical axis is time  $t \in [0, 1]$ . Colors denote the magnitude of the pointwise error, with red indicating larger discrepancy and blue indicating near-zero error.

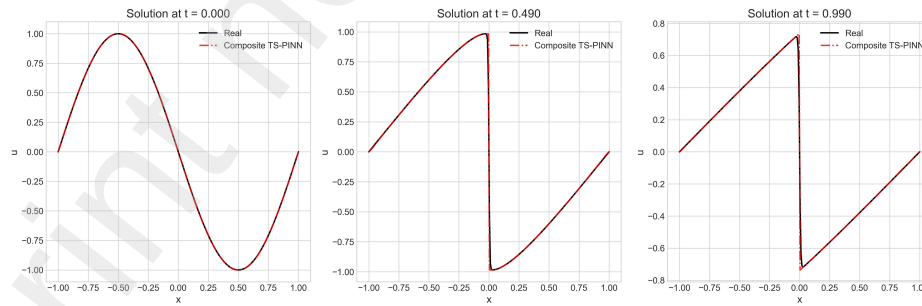


Figure 9: Comparison of real (black line) and TS-PINN (red dashed line) solutions for a one-dimensional PDE at three distinct times:  $t = 0.000$ ,  $0.490$ , and  $0.990$ .

| Equation         | Model   | MSE                   | MAE                   |
|------------------|---------|-----------------------|-----------------------|
| KdV Equation     | PINN    | $6.03 \times 10^{-4}$ | $1.39 \times 10^{-2}$ |
|                  | TS-PINN | $2.81 \times 10^{-4}$ | $1.13 \times 10^{-2}$ |
| Heat Equation    | PINN    | $2.17 \times 10^{-5}$ | $3.99 \times 10^{-3}$ |
|                  | TS-PINN | $1.11 \times 10^{-5}$ | $2.85 \times 10^{-3}$ |
| Burgers Equation | PINN    | $2.85 \times 10^{-4}$ | $8.78 \times 10^{-3}$ |
|                  | TS-PINN | $2.09 \times 10^{-4}$ | $2.21 \times 10^{-3}$ |

Table 1: Comparison of PINN and TS-PINN performance on KdV equation, heat equation and Burgers equation.

#### 5.3.4. Gray-Scott Equations

This experiment aims to solve the 2D Gray-Scott reaction-diffusion system using a staged PINN that incorporates a TS-PINN approach in its final training phase. The Gray-Scott model describes the interaction between two chemical species,  $u$  and  $v$ , and is defined by the following system of coupled nonlinear partial differential equations:

$$\frac{\partial u}{\partial t} = D_u \nabla^2 u - uv^2 + F(1 - u) \quad (5.7)$$

$$\frac{\partial v}{\partial t} = D_v \nabla^2 v + uv^2 - (F + K)v \quad (5.8)$$

where  $u(x, y, t)$  and  $v(x, y, t)$  are the concentrations of the two species. The parameters  $D_u$  and  $D_v$  are the diffusion coefficients,  $F$  is the feed rate, and  $K$  is the kill rate. The term  $\nabla^2$  is the Laplacian operator in two spatial dimensions,  $\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ .

The Gray-Scott system is a canonical model for studying pattern formation and is widely used in theoretical biology and chemistry to model Turing patterns. Depending on the parameters, it can generate a rich variety of complex, life-like structures, including spots, stripes, and self-replicating patterns. Its ability to produce these intricate dynamics makes it an excellent and challenging benchmark for evaluating the performance of numerical methods and PINNs in solving complex nonlinear systems.

In the experiments for the Gray-Scott equation, we employ a feedforward neural network with four hidden layers, each consisting of 128 neurons. The hyperbolic tangent (tanh) activation function is used in the hidden layers, while the Sigmoid activation function is adopted in the output layer. Training is performed

using the Adam optimizer with a learning rate of  $10^{-3}$  in Stage 0 and  $5 \times 10^{-4}$  in Stage 2 and Stage 3 through fine-tuning. By “fine-tuning”, we refer to adjusting the learning rate across different training stages, which is necessary to ensure stable and reliable convergence for both PINNs and TS-PINNs when applied to the Gray–Scott equation, a system known to exhibit stiff behavior and multiscale dynamics.

The experiments are conducted on the spatial domain  $[0, 1] \times [0, 1]$  with periodic boundary conditions. The training process focuses on the time interval  $t \in [2500, 3000]$ . The initial condition at  $t = 2500$  is taken from a pre-existing numerical simulation, providing a complex pattern for the model to learn and evolve. The numbers of initial, boundary, and interior collocation points are set to 10000, 400, and 10000, respectively, all sampled uniformly within their corresponding domains. In addition, 10000 validation points are selected uniformly over the spatio-temporal domain using an equidistant grid.

In the loss function, the weights associated with the differential equation residual, the initial condition, and the boundary conditions are set to 1, 100, and 1, respectively. The weight corresponding to the first-order temporal derivative term is chosen as 0.1.

For TS-PINNs, the training process consists of 5000 epochs for Stage 0, 20000 epochs for Stage 1, and 15000 epochs for Stage 2. For comparison, the standard PINN is trained for 5000 epochs in Stage 0 followed by 35000 epochs in Stage 1. The numerical solution of the Gray–Scott equation computed using the finite element method is used as the reference (ground-truth) solution throughout the experiments.

| Equation            | Model                | MSE                   | MAE                   |
|---------------------|----------------------|-----------------------|-----------------------|
| Gray-Scott Equation | PINN                 | $7.77 \times 10^{-5}$ | $6.64 \times 10^{-3}$ |
|                     | TS-PINN (1-st order) | $3.95 \times 10^{-5}$ | $4.65 \times 10^{-3}$ |
|                     | TS-PINN (2-nd order) | $1.59 \times 10^{-5}$ | $3.02 \times 10^{-3}$ |

Table 2: Comparison of PINN and TS-PINN performance on KdV equation, heat equation and Burgers equation.

The staged training strategy, when combined with a higher-capacity neural network and a strongly weighted data loss term, proves to be particularly effective for this problem. The pre-training stage on the initial state at  $t = 2500$  enables the model to establish a reliable baseline approximation. In the subsequent stages, this initial approximation is progressively refined by enforcing the governing PDE constraints together with their temporal derivative information.

This hierarchical training procedure enables the model to accurately capture the complex spatio-temporal patterns inherent in the Gray–Scott system, as illustrated in Figures 11 and 12. For the Gray–Scott equations, we conduct experiments with both  $k = 1$  and  $k = 2$  in (3.7) to investigate the effect of incorporating higher-order temporal derivative information. The TS-PINN model that includes only the first-order time-derivative loss term is referred to as the *first-order TS-PINN*, while the TS-PINN model that incorporates both first- and second-order time-derivative loss terms is referred to as the *second-order TS-PINN*.

The quantitative results reported in Table 2 and Figures 10, 11, and 12 demonstrate that TS-PINNs consistently achieve higher solution accuracy than standard PINNs. Moreover, the second-order TS-PINNs consistently outperform the first-order TS-PINNs, highlighting the benefit of incorporating higher-order derivative constraints.

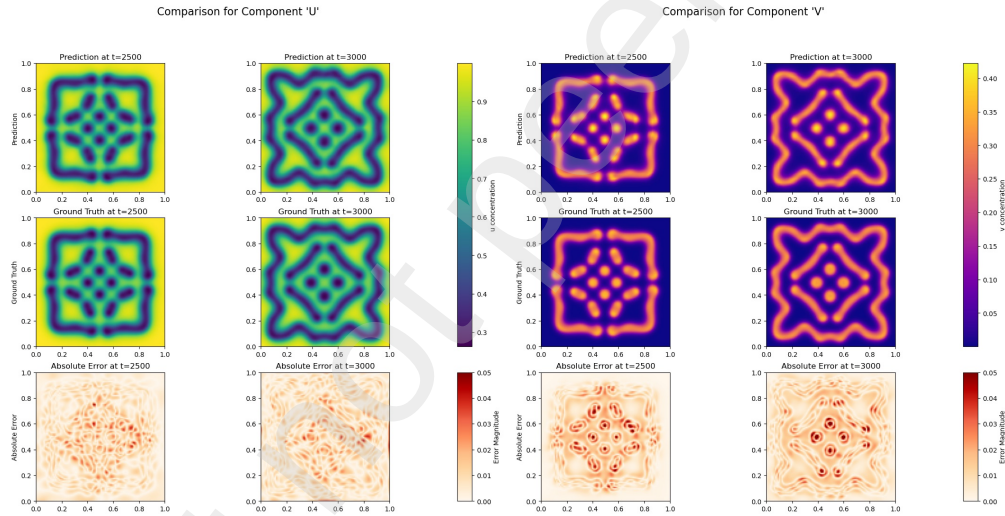


Figure 10: Comparison of the **standard PINN** prediction, ground truth, and point-wise error for the  $u, v$  component at different time snapshots.

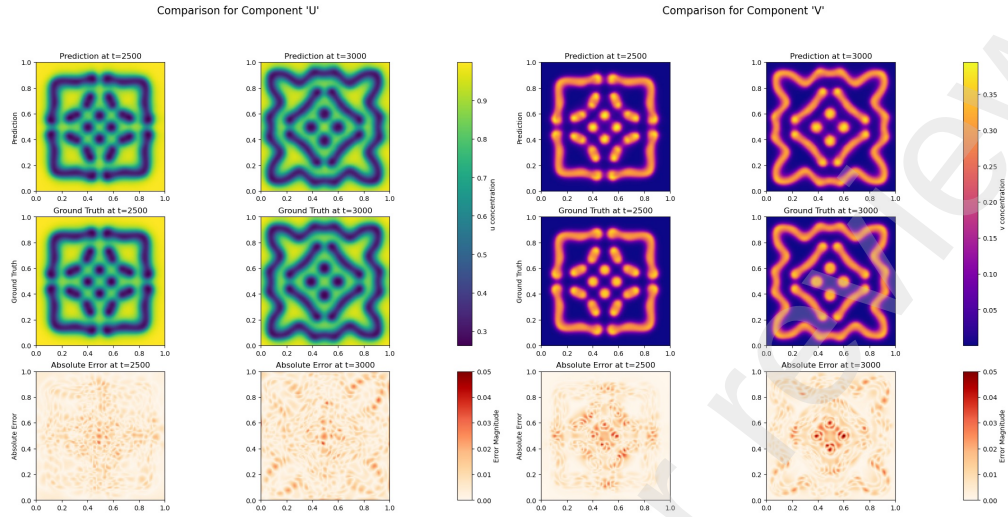


Figure 11: Comparison of the **first order** TS-PINN prediction, ground truth, and point-wise error for the  $u, v$  component at different time snapshots. The model accurately captures the evolution of the pattern.

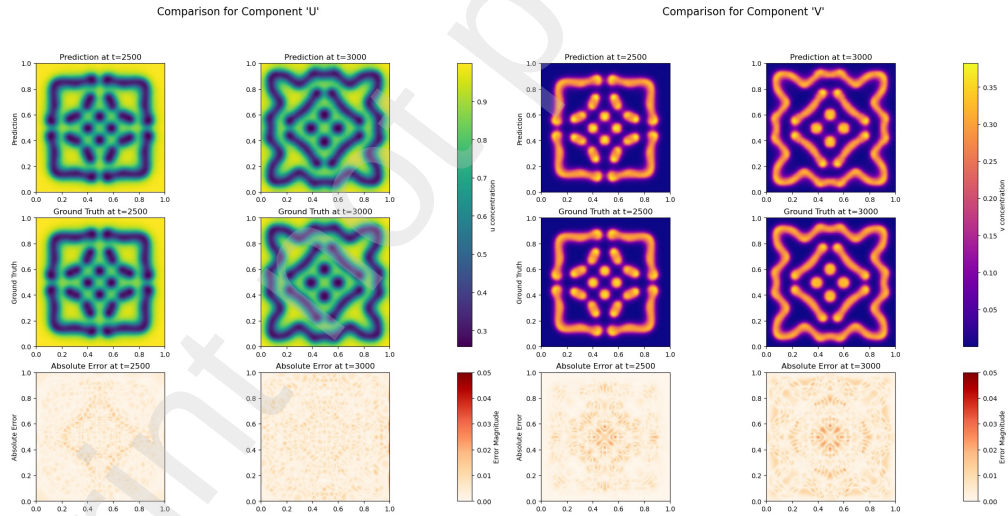


Figure 12: Comparison of the **second order** TS-PINN prediction, ground truth, and point-wise error for the  $u, v$  component at different time snapshots. The model accurately captures the evolution of the pattern.

We conducted 20 independent experiments to compare two training strategies for the TS-PINN model:

1. training with the total loss using the same total number of epochs as the three-stage strategy, and
2. the proposed three-stage training strategy. The statistical comparison of the resulting errors is shown in Figure 13.

From the figure, it can be observed that the three-stage training strategy achieves a slightly lower overall error than training the TS-PINN with the total loss over the same total number of epochs. Moreover, the error distribution produced by the three-stage training is more concentrated, indicating reduced variance across repeated experiments. These results demonstrate that the staged training strategy not only improves accuracy but also enhances the stability and robustness of the TS-PINN training process.

In addition, the average training time of the three-stage TS-PINN is 1720 s, whereas training the TS-PINN with the total loss for the same total number of epochs requires 4098 s. This significant reduction in computational cost clearly illustrates the superior training efficiency of the proposed staged training strategy.

In conclusion, the staged PINN methodology demonstrates a strong capability to solve the complex, nonlinear dynamics of the Gray-Scott system. The results indicate that by carefully balancing the influence of the data-driven and physics-informed loss components, and by providing the network with sufficient capacity, it is possible to achieve high-fidelity predictions that match the true solution.

## 6. Conclusion

In this paper, we introduced the Temporal Sobolev Physics-informed Neural Network (TS-PINN), a novel extension of the traditional Physics-informed Neural Network (PINN). By incorporating high-order temporal derivatives directly into the loss function, TS-PINNs provide a more accurate and robust approach to solving differential equations, particularly for problems where high-order derivatives play a critical role in capturing the system's dynamics.

The TS-PINN framework leverages the concept of Sobolev spaces, ensuring smoothness and better approximation of both the solution and its derivatives over time. This method enhances the training process by reducing errors, where traditional PINNs may fail to capture essential high-order dynamics. We demonstrated that by using TS-PINNs, it is possible to achieve a higher degree of accuracy compared to classical methods, especially for systems governed by complex differential equations.

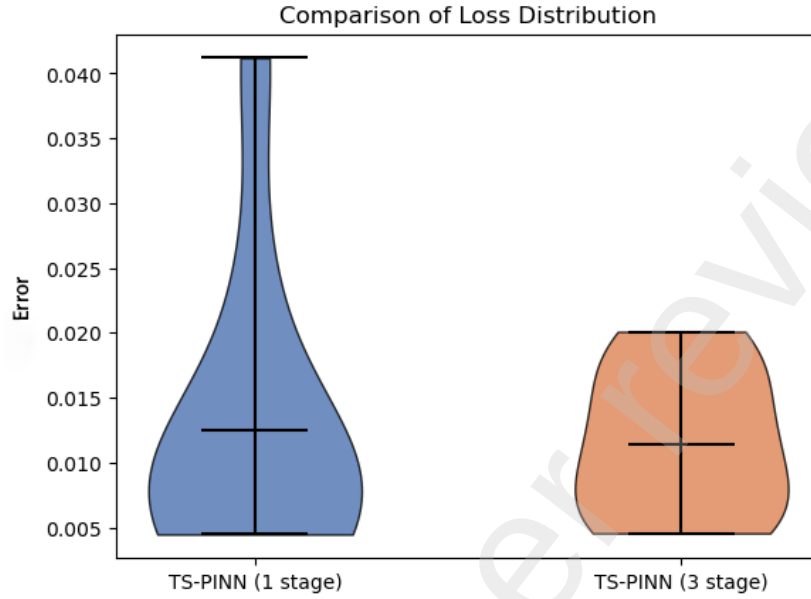


Figure 13: Distribution of errors for TS-PINN with single-stage and three-stage training. Both approaches achieve relative errors on the order of  $10^{-2}$ , indicating that TS-PINNs are able to learn a reasonably accurate solution in both cases. The three-stage TS-PINN exhibits a noticeably narrower violin shape, especially in the upper tail, indicating a more concentrated error distribution. This suggests that staged training reduces large local errors and leads to more uniform accuracy across the domain.

We also introduced an efficient training strategy for TS-PINNs, addressing the challenges associated with high-order derivatives and the computational complexity that arises in traditional numerical methods. The proposed loss function, built around temporal Sobolev norms, enables the network to not only model the solution accurately but also ensure precise modeling of its derivatives. This represents a significant step forward in utilizing neural networks for solving complex, real-world differential equations.

Future work will focus on further improving the efficiency of TS-PINNs, especially for large-scale problems with high-order dynamics, and exploring additional applications to more complex systems. Additionally, the development of adaptive schemes for varying derivative orders and improved error estimation techniques will continue to enhance the versatility and robustness of TS-PINNs in solving a wide range of physical and engineering problems.

## **7. Acknowledgment**

Hong-Kun Zhang is partially supported by Guangdong Basic and Applied Basic Research Foundation 2025A1515011952.